

Faculty of Information Technology (FIT) – Ho Chi Minh City University of Science (HCMUS)

CS423 / CSC13003 – Software Testing (AI-augmented · 2026)

AI POLICY · TEMPLATES — 2026 v1.0

AI Audit Report — 5-section Template per Artifact

Mandatory appendix for every AI-assisted homework (HW#01–HW#06, and Seminar).

Adapted from Med Kharbach, PhD (2026) — AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0. This adaptation is prepared for FIT@HCMUS – CS423 / CSC15003 Software Testing course.

1. Student Information

Field	Value
Student name (printed):	LÊ GIA BẢO
Student ID:	23127325
Class / Cohort:	KIEM THU PHAN MEM - 23KTPM2
Assignment ID (e.g., HW#00, HW#02):	HW#06
Assignment date:	30/8/2026
AI tool(s) used:	CHATGPT,CURSOR
AI tool(s) used:	☒ Yes ☐ No

2. Instructions (read before filling)

- Add one row per AI-generated artifact (test case, script, checklist, OpenAPI spec, JMeter plan, etc.).
- Paste the verbatim prompt — DO NOT paraphrase.
- Paste the verbatim AI output (or include a labelled screenshot in the report).
- Tag the verdict: VALID / INVALID / INCOMPLETE.
- Reasoning must cite a course slide, ISTQB section, or technical RFC.
- Show the corrected artifact with the change highlighted.
- Sample rows are in italic — replace them before submission.

3. Audit Table — one row per artifact

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Sample (italic) — replace before submission:				
Tool: Cursor AI Time: 2026-08-22	Generated 45 test cases for Cart API (POST /api/cart).	INCOMPL ETE	Added TC-FP-031 to explicitly verify the API behavior when a valid Authorization header is provided. The test was added	Added TC-FP-031 to explicitly verify the API behavior when a valid Authorization header is provided. The test was added

20:27:00 (UTC+7) Prompt: 1. Generate with AI. Provide the SUT's API specification to an AI tool and drive it – step by step, not with a single generic prompt – to generate test cases for the API (target >= 30 per API). The cases must cover: domain partitions on every parameter (e.g., email format, password complexity, price > 0), state transitions (FR-10: pending → confirmed → shipping → delivered, plus cancelation rules), security (SEC-01-SEC-07, e.g., SQL	File created: cart.md FULL OUTPUT (cart.md content): # Test Cases – Cart API (FR-05) **Endpoint:** `POST /api/cart` **Base URL:** `http://localh ost:3000` **Auth Required:** Yes (Bearer Token) ## API Specification **Request Body:** ``json {   "id": 1,   "name": "Product Name",   "price": 99.99,   "quantity": 2 } ``		manually to improve security-related coverage and ensure that authentication behavior is evaluated separately from input validation. format edges.	manually to improve security-related coverage and ensure that authentication behavior is evaluated separately from input validation.
--	--	--	---	---

injection,
IDOR, role
escalation),
and schema
validation (the
response
shape exactly
matches the
spec). for POST
/api/cart

```
**Response:**
(Success - 200
OK)
```json
{
 "message":
 "Item added to
 cart",
 "cartItem":
 {
 "id": 1,
 "name":
 "Product
 Name",
 "price":
 99.99,

 "quantity": 2
 }
}
```

---

## Test Case
List

###
TC-CART-001:
Valid Cart Add
Request
-
**Priority:**
P0 (Critical)
-
**Category:**
Happy Path
-
**Description:
```

| | | | | |
|--|--|--|--|--|
| | <pre>** Valid product with all fields should be added to cart successfully - **Request Body:** ``json { "id": 1, "name": "iPhone 15 Pro", "price": 999.99, "quantity": 1 } ... - **Expected Response:** 200 OK with "Item added to cart" message - **Test Data:** Valid integer id, valid string name, positive price, positive integer quantity ### TC-CART-002: Price Zero</pre> | | | |
|--|--|--|--|--|

```
-
**Priority:**
P1 (High)
-
**Category:**
Domain
Partition -
Price
-
**Description:
** Price of 0
should be
handled
according to
policy
- **Request
Body:**
  ```json
 {
 "id": 2,
 "name":
"Free Sample",
 "price":
0,

 "quantity": 1
 }
  ```
- **Expected
Response:**
200 OK (free
items allowed)
or 400 (price
must be > 0)
- **Business
Rule:**
Depends on
cart policy
for zero-price
items
```

```
###
TC-CART-003:
Negative Price
-
**Priority:**
P0 (Critical)
-
**Category:**
Domain
Partition -
Price
-
**Description:**
** Negative
price should
be rejected
- **Request
Body:**
```json
{
 "id": 3,
 "name":
"Discount
Item",
 "price":
-10.00,

"quantity": 1
}
```
- **Expected
Response:**
400 Bad
Request with
validation
error
- **Test
Data:** price
< 0
```

```
###
TC-CART-004:
Very Large
Price (Exceeds
Maximum)
-
**Priority:**
P1 (High)
-
**Category:**
Domain
Partition -
Price
-
**Description:
** Price
exceeding
system maximum
should be
rejected
- **Request
Body:**
```json
{
 "id": 4,
 "name":
"Expensive
Item",
 "price":
999999999.99,

"quantity": 1
}
```
- **Expected
Response:**
400 Bad
Request with
```

```
validation
error

###
TC-CART-005:
Price with Too
Many Decimal
Places
-
**Priority:**
P2 (Medium)
-
**Category:**
Domain
Partition -
Price
-
**Description:
** Price with
more than 2
decimal places
should be
handled
- **Request
Body:**
```json
{
 "id": 5,
 "name":
"Precise
Item",
 "price":
10.999,

"quantity": 1
}
```
- **Expected
Response:**
200 OK
```

```
(truncated) or
400
(validation
error)

###
TC-CART-006:
Negative
Quantity
-
**Priority:**
P0 (Critical)
-
**Category:**
Domain
Partition -
Quantity
-
**Description:
** Negative
quantity
should be
rejected
- **Request
Body:**
```json
{
 "id": 6,
 "name":
"Test
Product",
 "price":
50.00,

"quantity": -5
}
```
- **Expected
Response:**
400 Bad
```

| | | | | |
|--|--|--|--|--|
| | <pre>Request with validation error ### TC-CART-007: Zero Quantity - **Priority:** P1 (High) - **Category:** Domain Partition - Quantity - **Description: ** Zero quantity should be rejected or ignored - **Request Body:** ```json { "id": 7, "name": "Test Product", "price": 50.00, "quantity": 0 } ... - **Expected Response:** 400 Bad Request or</pre> | | | |
|--|--|--|--|--|

| | | | | |
|--|---|--|--|--|
| | <pre>quantity treated as invalid ### TC-CART-008: Very Large Quantity - **Priority:** P1 (High) - **Category:** Domain Partition - Quantity - **Description: ** Quantity exceeding stock limit should be handled - **Request Body:** ``json   {     "id": 8,     "name": "Bulk Product",     "price": 10.00,  "quantity": 999999   }   `` - **Expected Response:**</pre> | | | |
|--|---|--|--|--|

```
400 Bad
Request or 200
with capped
quantity

###
TC-CART-009:
Quantity as
Decimal/Float
-
**Priority:**
P1 (High)
-
**Category:**
Domain
Partition -
Quantity
-
**Description:
** Decimal
quantity
should be
rejected
(integers
only)
- **Request
Body:**
```json
{
 "id": 9,
 "name":
"Test
Product",
 "price":
25.00,

"quantity":
1.5
}
```
```

```
- **Expected
Response:**
400 Bad
Request
(quantity must
be integer)

###
TC-CART-010:
Empty Product
ID
-
**Priority:**
P1 (High)
-
**Category:**
Domain
Partition - ID
-
**Description:
** Empty ID
should be
rejected
- **Request
Body:**
```json
{
 "id": "",
 "name":
"Test
Product",
 "price":
25.00,

"quantity": 1
}
```
- **Expected
Response:**
400 Bad
```

| | | | | |
|--|--|--|--|--|
| | <pre>Request with validation error ### TC-CART-011: Null Product ID - **Priority:** P1 (High) - **Category:** Domain Partition - ID - **Description: ** Null ID should be rejected - **Request Body:** ```json { "id": null, "name": "Test Product", "price": 25.00, "quantity": 1 } ``` - **Expected Response:** 400 Bad Request with</pre> | | | |
|--|--|--|--|--|

| | | | | |
|--|--|--|--|--|
| | <pre>validation error ### TC-CART-012: Non-Numeric ID - **Priority:** P1 (High) - **Category:** Domain Partition - ID - **Description: ** Non-numeric ID should be rejected - **Request Body:** ```json { "id": "abc123", "name": "Test Product", "price": 25.00, "quantity": 1 } ... - **Expected Response:** 400 Bad Request with validation error</pre> | | | |
|--|--|--|--|--|

| | | | | |
|--|--|--|--|--|
| | <pre>### TC-CART-013: Empty Product Name - **Priority:** P1 (High) - **Category:** Domain Partition - Name - **Description:** ** Empty name should be rejected - **Request Body:** ```json { "id": 10, "name": "", "price": 25.00, "quantity": 1 } ``` - **Expected Response:** 400 Bad Request with validation error ### TC-CART-014:</pre> | | | |
|--|--|--|--|--|

| | | | | |
|--|--|--|--|--|
| | <pre>Null Product Name - **Priority:** P1 (High) - **Category:** Domain Partition - Name - **Description: ** Null name should be rejected - **Request Body:** ```json { "id": 11, "name": null, "price": 25.00, "quantity": 1 } ``` - **Expected Response:** 400 Bad Request with validation error ### TC-CART-015: Very Long Product Name</pre> | | | |
|--|--|--|--|--|

[illegible]

```
"quantity": 1
  }
  ...
- **Expected
Response:**
200 OK
(truncated) or
400 (exceeds
max length)

###
TC-CART-016:
XSS Payload in
Product Name
-
**Priority:**
P0 (Critical)
-
**Category:**
Security - XSS
(SEC-02)
-
**Description:**
** XSS payload
in name should
be sanitized
- **Request
Body:**
```json
{
 "id": 13,
 "name":
"<script>alert
('XSS')</scrip
t>",
 "price":
25.00,
"quantity": 1
```

```
}
...
- **Expected
Response:**
400 Bad
Request or
sanitized
input (no
script
execution)

TC-CART-017:
SQL Injection
in Product
Name
-
Priority:
P0 (Critical)
-
Category:
Security - SQL
Injection
(SEC-01)
-
Description:
** SQL
injection in
name should be
sanitized
- **Request
Body:**
```json  
{  
  "id": 14,  
  "name":  
"Product";  
DROP TABLE  
cart;--",
```

```
        "price":
25.00,

    "quantity": 1
    }
    ...
- **Expected
Response:**
400 Bad
Request (no
SQL execution)

###
TC-CART-018:
SQL Injection
in ID Field
-
**Priority:**
P0 (Critical)
-
**Category:**
Security - SQL
Injection
(SEC-01)
-
**Description:
** SQL
injection in
ID should be
sanitized
- **Request
Body:**
```json
{
 "id": "1
OR 1=1",
 "name":
"Test
Product",
```

```
 "price":
25.00,

 "quantity": 1
 }
 ...
- **Expected
Response:**
400 Bad
Request (no
SQL execution)

###
TC-CART-019:
Missing ID
Field
-
Priority:
P1 (High)
-
Category:
Schema
Validation
-
**Description:
** Request
without ID
field should
be rejected
- **Request
Body:**
```json
{
    "name":
"Test
Product",
    "price":
25.00,

    "quantity": 1
```

```
}  
...  
  
- **Expected  
Response:**  
400 Bad  
Request with  
validation  
error  
  
###  
TC-CART-020:  
Missing Name  
Field  
-  
**Priority:**  
P1 (High)  
-  
**Category:**  
Schema  
Validation  
-  
**Description:**  
** Request  
without name  
field should  
be rejected  
- **Request  
Body:**  
``json  
{  
  "id": 15,  
  "price":  
25.00,  
  
"quantity": 1  
}  
...  
  
- **Expected  
Response:**  
400 Bad
```

	<pre>Request with validation error ### TC-CART-021: Missing Price Field - **Priority:** P1 (High) - **Category:** Schema Validation - **Description:** ** Request without price field should be rejected - **Request Body:** ```json { "id": 16, "name": "Test Product", "quantity": 1 } ``` - **Expected Response:** 400 Bad Request with validation error</pre>			
--	--	--	--	--

```
###
TC-CART-022:
Missing
Quantity Field
-
**Priority:**
P1 (High)
-
**Category:**
Schema
Validation
-
**Description:**
** Request
without
quantity field
should be
rejected
- **Request
Body:**
```json
{
 "id": 17,
 "name":
"Test
Product",
 "price":
25.00
}
...
- **Expected
Response:**
400 Bad
Request with
validation
error

###
TC-CART-023:
Extra Unknown
```

```
Field in
Request
-
Priority:
P2 (Medium)
-
Category:
Schema
Validation
-
Description:
** Extra
fields should
be ignored or
cause
validation
error
- **Request
Body:**
```json
{
  "id": 18,
  "name":
"Test
Product",
  "price":
25.00,

"quantity": 1,

"extraField":
"shouldBeIgnor
ed"
}
...
- **Expected
Response:**
200 OK (extra
fields
ignored) or
```

	<pre>400 (strict mode) ### TC-CART-024: No Authentication Token - **Priority:** P0 (Critical) - **Category:** Security - Authentication (SEC-03) - **Description: ** Request without auth token should be rejected - **Headers:** No `Authorization ` header - **Expected Response:** 401 Unauthorized ### TC-CART-025: Invalid Authentication Token Format - **Priority:** P0 (Critical)</pre>			
--	---	--	--	--

```
-  
**Category:**  
Security -  
Authentication  
(SEC-03)  
-  
**Description:**  
** Request  
with malformed  
token should  
be rejected  
- **Headers:**  
`Authorization`  
:  
InvalidToken`  
- **Expected  
Response:**  
401  
Unauthorized  
  
###  
TC-CART-026:  
Expired  
Authentication  
Token  
-  
**Priority:**  
P0 (Critical)  
-  
**Category:**  
Security -  
Authentication  
(SEC-03)  
-  
**Description:**  
** Request  
with expired  
token should  
be rejected
```

```
- **Headers:**
`Authorization
: Bearer
<expired_token
>`
- **Expected
Response:**
401
Unauthorized

###
TC-CART-027:
SQL Injection
via Price
Parameter
-
**Priority:**
P0 (Critical)
-
**Category:**
Security - SQL
Injection
(SEC-01)
-
**Description:
** SQL
injection via
price should
be sanitized
- **Request
Body:**
```json
{
 "id": 19,
 "name":
"Test
Product",
 "price":
"50; DROP
```

```
TABLE
users;--",

"quantity": 1
}
...

- **Expected
Response:**
400 Bad
Request (type
coercion or
validation)

###
TC-CART-028:
SQL Injection
via Quantity
Parameter
-
Priority:
P0 (Critical)
-
Category:
Security - SQL
Injection
(SEC-01)
-
Description:
** SQL
injection via
quantity
should be
sanitized
- **Request
Body:**
```json
{
  "id": 20,
```

```
    "name":
    "Test
    Product",
    "price":
    25.00,

    "quantity":
    "1; DELETE
    FROM cart;--"
    }
    ...

- **Expected
Response:**
400 Bad
Request (type
coercion or
validation)

###
TC-CART-029:
Price as
String Instead
of Number
-

**Priority:**
P1 (High)
-

**Category:**
Domain
Partition -
Price
-

**Description:
** String
price should
be rejected
- **Request
Body:**
```json
{
```

```
 "id": 21,
 "name":
"Test
Product",
 "price":
"fifty",

"quantity": 1
 }
 ...

- **Expected
Response:**
400 Bad
Request with
type error

###
TC-CART-030:
Quantity as
String Instead
of Number
-
Priority:
P1 (High)
-
Category:
Domain
Partition -
Quantity
-
**Description:
** String
quantity
should be
rejected
- **Request
Body:**
```json
{
    "id": 22,
```

```
        "name":
"Test
Product",
        "price":
25.00,

"quantity":
"one"
    }
    ...

- **Expected
Response:**
400 Bad
Request with
type error

###
TC-CART-031:
Invalid JSON
Format
-
**Priority:**
P1 (High)
-
**Category:**
Schema
Validation
-
**Description:
** Malformed
JSON should be
rejected
- **Request
Body:** `{id:
23, name:
"Test", price:
25, quantity:
1}`
- **Expected
Response:**
```

	<pre>400 Bad Request (JSON parsing error) ### TC-CART-032: Empty Request Body - **Priority:** P1 (High) - **Category:** Schema Validation - **Description:** ** Empty body should be rejected - **Request Body:** `{}` - **Expected Response:** 400 Bad Request with validation error ### TC-CART-033: Non-Existent Product ID - **Priority:** P1 (High) - **Category:** Business Logic</pre>			
--	---	--	--	--

```
-  
**Description:  
** Adding  
non-existent  
product should  
be handled  
- **Request  
Body:**  
  ``json  
  {  
    "id":  
999999,  
    "name":  
"Non-existent  
Product",  
    "price":  
25.00,  
  
    "quantity": 1  
  }  
  ``  
- **Expected  
Response:**  
200 OK (added  
anyway) or 404  
(product not  
found)  
  
###  
TC-CART-034:  
Adding Same  
Product Twice  
-  
**Priority:**  
P1 (High)  
-  
**Category:**  
State  
Transition -  
Cart State
```

```
-
**Description:
** Adding
existing
product should
update
quantity
- **Request
1:**
  ``json
  {
    "id": 24,
    "name":
"Test
Product",
    "price":
25.00,

"quantity": 2
  }
  ...
- **Request
2:**
  ``json
  {
    "id": 24,
    "name":
"Test
Product",
    "price":
25.00,

"quantity": 3
  }
  ...
- **Expected
Response:**
Quantity
should be
summed or
```

```
overwritten
(verify
behavior)

###
TC-CART-035:
Price Mismatch
with Server
-
**Priority:**
P2 (Medium)
-
**Category:**
Business Logic
-
**Description:**
** Client
price differs
from server
price
- **Request
Body:**
```json
{
 "id": 25,
 "name":
"Product A",
 "price":
10.00,

"quantity": 1
}
...
- **Expected
Response:**
200 OK (server
may use its
own price) or
error if
```

	<pre>strict validation  ### TC-CART-036: Unicode Characters in Product Name - **Priority:** P2 (Medium) - **Category:** Domain Partition - Name - **Description: ** Unicode in name should be handled - **Request Body:** ```json {   "id": 26,   "name": "Sản Phẩm Tiếng Việt",   "price": 25.00,  "quantity": 1 } ``` - **Expected Response:** 200 OK or 400 (depending on</pre>			
--	------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	<pre>Unicode support)  ### TC-CART-037: Duplicate Cart Item (Same ID, Different Name) - **Priority:** P2 (Medium) - **Category:** Business Logic - **Description: ** Same ID with different name should be handled - **Request Body:** ```json {   "id": 27,   "name": "Different Name",   "price": 25.00,  "quantity": 1 } ``` - **Expected Response:** 200 OK (behavior</pre>			
--	---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	depends on server logic)			
Tool: Cursor AI   Time: 2026-08-22 20:34:00 (UTC+7)   Prompt:    <pre>1. Generate with AI. Provide the SUT's API specification to an AI tool and drive it – step by step, not with a single generic prompt – to generate test cases for the API (target &gt;= 30( per API). The cases must cover: domain partitions on every parameter (e.g., email format, password complexity, price &gt; 0), state transitions (FR-10: pending → confirmed → shipping → delivered, plus</pre>	<pre>Output:   Generated 30 test cases for Cancel Order API (PUT /api/orders/:id/cancel). File created: cancel-order.md  FULL OUTPUT (cancel-order.md content):  # Test Cases – Cancel Order API (FR-10)  **Endpoint:** `PUT /api/orders/:id/cancel`  **Base URL:** `http://localhost:3000`  **Auth Required:** Yes (Bearer Token)  ## API Specification</pre>	Incomplete	INCOMPLETE — The AI provided broad coverage of state transitions, input partitions, authentication, security, and malformed requests. However, several cases use assumptions that are not established by the API specification, such as whether confirmed orders can be cancelled (TC-CANCEL-002), whether the operation is admin-only (TC-CANCEL-024), and whether rate limiting is implemented (TC-CANCEL-025). Some expected results are also overly broad, for example 400 or 404 and 200 or 400, which makes the expected behavior non-deterministic. In addition, cases such as XSS through an unspecified custom header (TC-CANCEL-019) and extra request fields (TC-CANCEL-023) are not justified by the endpoint contract. According to ISTQB	The student reviewed the generated cases against the actual API behavior and removed or revised cases that depended on unsupported assumptions. Expected results were changed from broad alternatives such as 400 or 404 to the status code actually defined or observed for the SUT. State-transition cases were aligned with the actual cancellation rules, while authentication and authorization cases were retained only where the API/SUT provided evidence for the behavior. Additional manually designed cases were then added to cover gaps identified during the review, particularly authorization-header behavior, HTTP method enforcement, and explicit response-schema validation.

<pre>cancelation rules), security (SEC-01-SEC-07, e.g., SQL injection, IDOR, role escalation), and schema validation (the response shape exactly matches the spec). for PUT /api/orders/:id /cancel</pre>	<pre>**Path Parameters:** - `id` - Order ID (integer)  **Response:** (Success - 200 OK) ```json {   "message": "Order cancelled successfully",   "order": {     "id": 1,     "status": "cancelled",  "updatedAt": "2026-08-22T10 :00:00Z"   } } ```  ---  ## Test Case List  ### TC-CANCEL-001: Cancel Pending Order (Valid State Transition) - **Priority:** P0 (Critical)</pre>		Foundation Level, Black-box Test Techniques, test conditions and expected results should be derived from specified behavior, while State Transition Testing requires clearly defined valid and invalid transitions. Therefore, the generated set provides useful coverage but requires human validation against the actual SUT and specification.	
-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--

```
-
Category:
State
Transition -
Valid
Cancellation
-
**Description:
** Order in
"pending"
status should
be cancellable
- **Request:**
`PUT
/api/orders/1/
cancel`
-
**Prerequisite
:** Order must
be in
"pending"
status
- **Expected
Response:**
200 OK with
order status
changed to
"cancelled"
- **State
Transition:**
pending →
cancelled

###
TC-CANCEL-002:
Cancel
Confirmed
Order
```

```
-
Priority:
P0 (Critical)
-
Category:
State
Transition -
Conditional
Cancellation
-
**Description:
** Order in
"confirmed"
status may be
cancellable
depending on
policy
- **Request:**
`PUT
/api/orders/2/
cancel`
-
**Prerequisite
:** Order must
be in
"confirmed"
status
- **Expected
Response:**
200 OK (if
allowed) or
400 (if not
allowed)

###
TC-CANCEL-003:
Attempt to
Cancel
Shipping Order
```

```
-
Priority:
P0 (Critical)
-
Category:
State
Transition -
Invalid
Cancellation
-
**Description:
** Order in
"shipping"
status should
NOT be
cancellable
- **Request:**
`PUT
/api/orders/3/
cancel`
-
**Prerequisite
:** Order must
be in
"shipping"
status
- **Expected
Response:**
400 Bad
Request with
"cannot
cancel" error
- **State
Transition:**
shipping →
cancelled
(BLOCKED)

###
TC-CANCEL-004:
```

	<pre>Attempt to Cancel Delivered Order - **Priority:** P0 (Critical) - **Category:** State Transition - Invalid Cancellation - **Description: ** Order in "delivered" status should NOT be cancellable - **Request:** `PUT /api/orders/4/ cancel` - **Prerequisite :** Order must be in "delivered" status - **Expected Response:** 400 Bad Request with "cannot cancel" error - **State Transition:** delivered →</pre>			
--	--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	<pre>cancelled (BLOCKED)  ### TC-CANCEL-005: Attempt to Cancel Already Cancelled Order - **Priority:** P1 (High) - **Category:** State Transition - Invalid Cancellation - **Description: ** Order already cancelled cannot be cancelled again - **Request:** `PUT /api/orders/5/ cancel` - **Prerequisite :** Order must be in "cancelled" status - **Expected Response:** 400 Bad Request with</pre>			
--	--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

```
"already
cancelled"
error

###
TC-CANCEL-006:
Cancel
Non-Existent
Order
-
Priority:
P0 (Critical)
-
Category:
Business Logic
- Non-Existent
Resource
-
Description:
** Cancelling
non-existent
order should
return
appropriate
error
- **Request:**
`PUT
/api/orders/99
9999/cancel`
- **Expected
Response:**
404 Not Found

###
TC-CANCEL-007:
Cancel Order
with Invalid
ID Format
(String)
```

```
-
Priority:
P1 (High)
-
Category:
Domain
Partition - ID
Validation
-
**Description:
** Non-numeric
order ID
should be
rejected
- **Request:**
`PUT
/api/orders/abc/cancel`
- **Expected
Response:**
400 Bad
Request with
validation
error

###
TC-CANCEL-008:
Cancel Order
with Negative
ID
-
Priority:
P1 (High)
-
Category:
Domain
Partition - ID
Validation
-
**Description:
```

```
** Negative
order ID
should be
rejected
- **Request:**
`PUT
/api/orders/-1
/cancel`
- **Expected
Response:**
400 Bad
Request with
validation
error

###
TC-CANCEL-009:
Cancel Order
with Zero ID
-
Priority:
P1 (High)
-
Category:
Domain
Partition - ID
Validation
-
**Description:
** Zero order
ID should be
rejected
- **Request:**
`PUT
/api/orders/0/
cancel`
- **Expected
Response:**
400 Bad
Request with
```

	<pre>validation error  ### TC-CANCEL-010: Cancel Order with Float ID - **Priority:** P1 (High) - **Category:** Domain Partition - ID Validation - **Description:** ** Floating-point order ID should be rejected - **Request:** `PUT /api/orders/1. 5/cancel` - **Expected Response:** 400 Bad Request with validation error  ### TC-CANCEL-011: Cancel Order with SQL Injection in ID</pre>			
--	----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

```
-
Priority:
P0 (Critical)
-
Category:
Security - SQL
Injection
(SEC-01)
-
**Description:
** SQL
injection in
order ID
should be
sanitized
- **Request:**
`PUT
/api/orders/1'
OR
'1'='1/cancel`
- **Expected
Response:**
400 Bad
Request (no
SQL execution)

###
TC-CANCEL-012:
Cancel Order
with UNION SQL
Injection in
ID
-
Priority:
P0 (Critical)
-
Category:
Security - SQL
Injection
(SEC-01)
```

```
-
**Description:
** UNION-based
SQL injection
should be
blocked
- **Request:**
`PUT
/api/orders/1
UNION SELECT *
FROM
users--/cancel
,
- **Expected
Response:**
400 Bad
Request (no
data exposure)

TC-CANCEL-013:
Cancel Order
Without
Authentication
-
Priority:
P0 (Critical)
-
Category:
Security -
Authentication
(SEC-03)
-
**Description:
** Request
without auth
token should
be rejected
- **Headers:**
No
```

<pre>`Authorization ` header - **Expected Response:** 401 Unauthorized  ### TC-CANCEL-014: Cancel Order with Invalid Token Format - **Priority:** P0 (Critical) - **Category:** Security - Authentication (SEC-03) - **Description: ** Request with malformed token should be rejected - **Headers:** `Authorization : InvalidToken` - **Expected Response:** 401 Unauthorized  ### TC-CANCEL-015: Cancel Order with Expired Token</pre>			
------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

```
-
Priority:
P0 (Critical)
-
Category:
Security -
Authentication
(SEC-03)
-
**Description:
** Request
with expired
token should
be rejected
- **Headers:**
`Authorization
: Bearer
<expired_token
>`
- **Expected
Response:**
401
Unauthorized

###
TC-CANCEL-016:
Cancel Another
User's Order
(IDOR)
-
Priority:
P0 (Critical)
-
Category:
Security -
IDOR (SEC-03)
-
**Description:
** User should
not be able to
```

```
cancel another
user's order
- **Request:**
 `PUT
 /api/orders/10
 0/cancel`
 (order belongs
 to different
 user)
- **Expected
 Response:**
 403 Forbidden
- **Security
 Check:**
 Verify
 authorization
 check exists

###
TC-CANCEL-017:
Cancel Order
with Basic
Auth Instead
of Bearer
-
Priority:
P1 (High)
-
Category:
Security -
Authentication
(SEC-03)
-
Description:
** Wrong auth
type should be
rejected
- **Headers:**
 `Authorization
```

```
: Basic
dXNlcjpwYXNz`
- **Expected
Response:**
401
Unauthorized

###
TC-CANCEL-018:
Cancel Order
Without Bearer
Prefix
-
Priority:
P1 (High)
-
Category:
Security -
Authentication
(SEC-03)
-
**Description:
** Token
without Bearer
prefix should
be rejected
- **Headers:**
`Authorization
:
some_token_wit
hout_bearer`
- **Expected
Response:**
401
Unauthorized

###
TC-CANCEL-019:
XSS Payload in
Custom Header
```

```
-
Priority:
P1 (High)
-
Category:
Security - XSS
(SEC-02)
-
**Description:
** XSS in
custom header
should be
sanitized
- **Headers:**
`X-Custom-Head
er:
<script>alert(
'XSS')</script
>`
- **Expected
Response:**
200 OK or
sanitized

###
TC-CANCEL-020:
Very Long
Order ID
-
Priority:
P2 (Medium)
-
Category:
Domain
Partition - ID
Validation
-
**Description:
** Extremely
long order ID
```

```
should be
handled
- **Request:**
`PUT
/api/orders/99
999999999999999
999999999999999/ca
ncel`
- **Expected
Response:**
400 Bad
Request or 404

###
TC-CANCEL-021:
Null Order ID
-
Priority:
P1 (High)
-
Category:
Domain
Partition - ID
Validation
-
**Description:
** Null order
ID should be
rejected
- **Request:**
`PUT
/api/orders/nu
ll/cancel`
- **Expected
Response:**
400 Bad
Request with
validation
error
```

```
###
TC-CANCEL-022:
Missing Order
ID
-
Priority:
P1 (High)
-
Category:
Schema
Validation
-
**Description:
** Missing
order ID
should return
404
- **Request:**
`PUT
/api/orders//c
ancel`
- **Expected
Response:**
404 Not Found
or 400

###
TC-CANCEL-023:
Extra Fields
in Request
Body
-
Priority:
P2 (Medium)
-
Category:
Schema
Validation
-
**Description:
```

	<pre>** Extra fields should be ignored or cause error - **Request:** `PUT /api/orders/1/ cancel` with body `{"extraField" : "value"}` - **Expected Response:** 200 OK (extra ignored) or 400 (strict mode)  ### TC-CANCEL-024: Cancel Order with Token for Different Role - **Priority:** P0 (Critical) - **Category:** Security - Role Escalation (SEC-05) - **Description:** ** User role token should not allow admin actions (if applicable)</pre>			
--	--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

```
- **Headers:**
`Authorization
: Bearer
<user_token>`
(not admin)
- **Request:**
`PUT
/api/orders/1/
cancel`
(admin-only
operation)
- **Expected
Response:**
403 Forbidden
(if role-based
access control
exists)

###
TC-CANCEL-025:
Rate Limiting
- Multiple
Rapid Cancel
Requests
-
Priority:
P1 (High)
-
Category:
Security -
Rate Limiting
(SEC-06)
-
**Description:
** Multiple
rapid cancel
attempts
should be rate
limited
```

	<pre>- **Request:** Send 20+ cancel requests in quick succession - **Expected Response:** 429 Too Many Requests after threshold  ### TC-CANCEL-026: Race Condition - Concurrent Cancellation - **Priority:** P1 (High) - **Category:** Security - Race Condition (SEC-07) - **Description:** ** Two simultaneous cancel requests should be handled atomically - **Request:** Send 2 identical cancel requests simultaneously</pre>			
--	------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

```
- **Expected
Response:**
One succeeds,
one gets
"already
cancelled"
error

###
TC-CANCEL-027:
Cancel Order
with Invalid
JSON
-
Priority:
P1 (High)
-
Category:
Schema
Validation
-
**Description:
** Invalid
JSON in
request body
should be
handled
- **Request:**
`PUT
/api/orders/1/
cancel` with
body
`{invalid}`
- **Expected
Response:**
200 OK or 400
(body may be
ignored)
```

	<pre>### TC-CANCEL-028: Cancel Order with Token Missing User Context - **Priority:** P1 (High) - **Category:** Security - Authorization - **Description: ** Token without user context should be rejected - **Headers:** `Authorization : Bearer &lt;token_without _user_id&gt;` - **Expected Response:** 401 Unauthorized  ### TC-CANCEL-029: Cancel Order While Status Transition is in Progress - **Priority:** P2 (Medium) - **Category:**</pre>			
--	------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	State   Transition –   Race Condition   –   <b>**Description:</b>   <b>** Cancel</b>   during status   update should   be handled   atomically   –   <b>**Prerequisite</b>   <b>:** Order</b>   status being   updated   simultaneously   – <b>**Expected</b>   <b>Response:**</b>   Consistent   state after   concurrent   operations   ###   TC-CANCEL-030:   Cancel Order   with Special   Characters in   ID   –   <b>**Priority:**</b>   P2 (Medium)   –   <b>**Category:**</b>   Domain   Partition – ID   Validation   –   <b>**Description:</b>   <b>** ID with</b>			
--	-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	<pre>special characters should be rejected - **Request:** `PUT /api/orders/10 #\$/cancel` - **Expected Response:** 400 Bad Request</pre>			
<pre>Tool: Cursor AI Time: 2026-08-28 21:50:00 (UTC+7) Prompt:       generate cho forgot-password đi, generate cả file md và json đồ nha.  (forgot-passwor d api chứ không phải reset-password đâu)</pre>	<pre>Generated test artifacts for the Forgot Password API (FR-03), specifically `POST /api/forgot-pa ssword`, not `POST /api/reset-pas sword`.  Source files consulted: - `eshop-sut/api _specification .md` - `eshop-sut/bac kend/server.js` - `eshop-sut/bac kend/database. js`</pre>	Incomplet e	<b>INCOMPLETE</b> — The AI correctly identified the target as POST <code>/api/forgot-password</code> , consulted the API specification and SUT source code, and generated both the Markdown test specification and Postman collection. It also provided useful coverage for input partitions, malformed requests, headers, security, and response schemas. However, several security-oriented cases require careful validation against the actual specification and SUT behavior. For example, the expected 429 Too Many Requests response for rate limiting assumes a	The student reviewed the generated Markdown and Postman collection against the API specification and the actual Forgot Password implementation. Cases with unsupported assumptions were revised so that expected results reflected the observed SUT behavior rather than assumed security controls. In particular, the rate-limiting case was treated as a security-gap observation because the backend does not implement rate limiting, rather than assuming that 429 is the required response. The account-enumeratio n case was

	<pre>API details identified: - Endpoint: `POST /api/forgot-pass ssword` - Request body: `{ "email": "test@domain.c om" }` - Auth required: No - Success response: `200 OK` with `message` and `resetToken` - Seeded valid emails: - `admin@eshop.c om` - `lgbao23@clc.f itus.edu.vn` - `test@eshop.co m`  Files created: - `forgot-passwo rd.md` - `FR3_Forgot_Pa ssword.postman</pre>		control that the backend does not implement. Similarly, security probes such as XSS, command injection, and newline injection should only be considered valid test conditions when their expected behavior is defined or justified by the security requirements. According to <b>ISTQB Foundation Level – Test Analysis and Design</b>, test conditions should be derived from test basis and risk, while expected results should be determinable from requirements or observable SUT behavior. Therefore, the artifact was useful but required human review before execution.	retained because the SUT actually returns 404 User not found for a non-existent email, making the behavior observable and testable. The generated collection was also manually reviewed to ensure that request methods, headers, status-code assertions, and response-schema checks matched the actual API.
--	----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

	<pre>_collection.js on` - `FR3_Local.pos tman_environme nt.json`  Generated content summary: - Covered valid seeded emails, invalid email format partitions, missing/null email, wrong JSON schema, malformed JSON, wrong HTTP methods, wrong or missing content type, SQL injection, XSS/HTML injection, command-like payloads, header/newline injection, account enumeration, rate limiting, and concurrent token-generati on probes.</pre>			
--	-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	<pre>- Added Postman collection variables and environment variables:     `baseUrl`,     `studentId`, and     `authToken`. - Added `X-Student-Id: {{studentId}}` header to every request. - Used one clear status-code assertion per test case, following the style of  `FR10_Cancel_O rder.postman_c ollection.json` . - Added schema checks for success and error response cases.  Verification performed: - Parsed `FR3_Forgot_Pa ssword.postman _collection.js</pre>			
--	-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

```
on`
successfully
as JSON.
 - Parsed
`FR3_Local.pos
tman_environme
nt.json`
successfully
as JSON.
 - Confirmed
the generated
collection
contains 45
request items.

Important
notes:
 - The
backend
currently
returns
`resetToken`
directly in
the
forgot-passwor
d
response.
This is useful
for testing
but should be
treated as a
potential
security
issue in a
real system.
 - The
backend
currently
returns `404
User not
```

	<pre> found` for non-existent email,     which may reveal whether an account exists. TC-FP-014 documents this as an  information-di sclosure/accou nt-enumeration test.     - The backend does not appear to implement forgot-passwor d rate limiting.     TC-FP-040 expects `429 Too Many Requests` to expose that missing control. </pre>			
<pre> Tool:Chat GPT Time: 13:00 29/8/2026 Prompt: give me 1 file md only Generate Skill.Md to generate test cases for the API (target ≥ </pre>	<pre> # SKILL.md - AI-Driven API Test Generator  ## 1. Skill Name  AI-Driven API Test Generator </pre>	Valid	VALID — The generated SKILL.md provides a structured and traceable process for API test generation. It requires the AI to first identify the selected endpoint, extract the API contract, and use	No correction was required. The student reviewed the generated SKILL.md and confirmed that it provides sufficient guidance for generating API test cases according to the assignment requirements. The skill explicitly

35 per API). The cases must cover: domain partitions on every parameter (e.g., email format, password complexity, price &gt; 0), state transitions (FR-10: pending → confirmed → shipping → delivered, plus cancelation rules), security (SEC-01-SEC-07, e.g., SQL injection, IDOR, role escalation), and schema validation (the response shape exactly matches the spec). , it read api specification, understand specification, udnerstand api contract, design testcases and output to testcases.md, then generate	<pre> ---  ## 2. Purpose  This skill generates comprehensive API test cases for **user-selected endpoints** from an API specification.  The skill does NOT automatically generate tests for every endpoint in the API specification.  The user provides:  1. An API specification. 2. One or more target endpoints.  The agent then:  1. Reads the API specification. </pre>		the specification as the source of truth. It also explicitly covers domain partitioning, boundary values, required/optional fields, security testing, state transitions where applicable, and success/error response-schema validation. The skill further requires duplicate removal, coverage validation, and final verification of each test case against the API contract. This is consistent with <b>ISTQB Foundation Level – Test Analysis and Design</b>, particularly the use of equivalence partitioning, boundary value analysis, state transition testing, and risk-based test analysis. Because the generated skill includes mechanisms to avoid unsupported assumptions and requires contract traceability, it was accepted as valid.	enforces a minimum of 35 test cases per selected API, requires coverage of applicable input partitions and boundaries, includes the specified security categories (SEC-01–SEC-07), validates response schemas, and prevents generation of tests for endpoints not explicitly selected by the user. The output requirements for testcases.md, tests.json, and the Postman Collection are also clearly defined. Therefore, the artifact was accepted as generated without modification.
------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

tests json collection and postman colleciton json	2. Finds the requested endpoints.   3. Understands the API specification for those endpoints.   4. Extracts the API contract.   5. Designs at least 35 test cases per selected endpoint.   6. Validates test coverage.   7. Generates <code>`testcases.md`</code>.   8. Generates <code>`tests.json`</code>.   9. Generates <code>`postman_collection.json`</code>.   ---   ## 3. Input   The skill accepts:   <code>```text</code>   API Specification   +   Target Endpoint(s)			
------------------------------------------------------------	---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

Example:

```
API
Specification:
openapi.yaml

Target
endpoints:
POST
/api/forgot-password
PUT
/api/orders/{id}/cancel
```

Another example:

```
API
Specification:
openapi.yaml

Target
endpoint:
PUT
/api/orders/{id}/cancel
```

The agent must only generate tests for the endpoint(s) explicitly provided

by the user.

---

## ## 4. Endpoint Selection

The user may provide:

Method + Path

Example:

```

POST

/api/forgot-password

```

or:

```

PUT

/api/orders/{id}/cancel

```

The endpoint must be matched against the API specification.

Matching should consider:

```

| | | | | |
|--|---|--|--|--|
| | <pre>HTTP method + endpoint path ``` Example: User input: ``` PUT /api/orders/{i d}/cancel ``` Specification: ``` /api/orders/{i d}/cancel: put: ... ``` The agent selects only this operation. ## 5. Endpoint Validation Before generating tests, verify that every requested endpoint exists in the</pre> | | | |
|--|---|--|--|--|

| | | | | |
|--|---|--|--|--|
| | <pre>API specification. For each requested endpoint: ... IF endpoint exists: Continue ELSE: Report: "Endpoint not found in API specification" Do not generate tests for that endpoint. ... Example: Requested: ... PUT /api/orders/{i d}/cancel ... Result: ...</pre> | | | |
|--|---|--|--|--|

| | | | | |
|--|---|--|--|--|
| | <pre>✓ Endpoint found ✓ HTTP method found ✓ API contract extracted ''' ## 6. Read API Specification Read the API specification as the source of truth. Only inspect the parts necessary to understand the selected endpoint, including: ''' HTTP method Path Operation ID Description Path parameters Query parameters Headers Request body Content type Request schema Required fields</pre> | | | |
|--|---|--|--|--|

| | | | | |
|--|---|--|--|--|
| | <div>Optional fields</div> <div>Data types</div> <div>Formats</div> <div>Minimum values</div> <div>Maximum values</div> <div>Minimum lengths</div> <div>Maximum lengths</div> <div>Patterns</div> <div>Enumerations</div> <div>Authentication</div> <div>Authorization</div> <div>Business rules</div> <div>Response status codes</div> <div>Response schemas</div> <div>Error schemas</div> <div>Response headers</div> <div>Examples</div> <div>...</div> <div>Do not generate tests for unrelated endpoints.</div> <div>## 7.</div> <div>Understand API Contract</div> <div>For each selected endpoint, create an internal API contract.</div> | | | |
|--|---|--|--|--|

| | | | | |
|--|--|--|--|--|
| | <pre>The contract should contain: ... Endpoint HTTP Method Path Parameters Query Parameters Headers Request Body Authentication Authorization Input Constraints Business Rules Success Responses Error Responses Response Schemas ... Example: ```json { "method": "PUT", "path": "/api/orders/{ id}/cancel", "parameters": { "id": {</pre> | | | |
|--|--|--|--|--|

```
"location":
  "path",

  "required":
    true,
    "type":
      "integer",

      "minimum": 1
    }
  },

  "authentication": true,
  "responses":
    {
      "200": {},
      "400": {},
      "401": {},
      "403": {},
      "404": {}
    }
  }
}
...

## 8. Test
Case Target

Generate:

...

>= 35 test
cases per
selected
endpoint
...
```

If multiple endpoints are provided:

```

Endpoint 1 →

>= 35 tests

Endpoint 2 →

>= 35 tests

Endpoint 3 →

>= 35 tests

```

Do not combine the test count across endpoints.

For example:

```

POST

/api/login

35 tests

PUT

/api/orders/{id}/cancel

35 tests

Total:

70 tests

```

9. Domain Partition

Testing

Domain

partition

| | | | | |
|--|---|--|--|--|
| | <p>testing is mandatory for every applicable parameter.</p> <p>For every parameter, identify valid and invalid partitions.</p> <p>Consider:</p> <p>...</p> <p>Valid values
Invalid values
Boundary values
Below minimum
Above maximum
Empty values
Null values
Missing values
Wrong data types
Invalid formats
Special characters
...</p> <p>Only use constraints defined by the API specification.</p> | | | |
|--|---|--|--|--|

```
## 10.
Parameter
Examples

### Email

For an email
parameter,
consider:

...

Valid email
Invalid email
format
Missing @
Missing local
part
Missing domain
Multiple @
Empty email
Null email
Missing email
Very long
email
Wrong data
type
...

Example:

...

test@example.c
om
invalid-email
@example.com
test@
test@@example.
com
""
null
```

```
### Password
```

```
If password  
constraints  
exist, cover:
```

```
Valid password  
Minimum valid  
length  
Maximum valid  
length  
Below minimum  
Above maximum  
Missing  
uppercase  
Missing  
lowercase  
Missing number  
Missing  
special  
character  
Empty password  
Null password  
Missing  
password  
Wrong data  
type
```

```
Do not invent  
password rules  
not defined by  
the  
specification.
```

```
### Numeric  
Parameters
```

For parameters
such as:

```

price > 0

```

cover:

```

Positive value

Minimum valid

value

Zero

Negative value

Maximum valid

value

Above maximum

Decimal

Null

Missing

String

Boolean

```

ID

Parameters

For IDs,
consider:

```

Valid existing

ID

Non-existing

ID

Minimum ID

Zero

Negative ID

	Decimal ID   Very large ID   String   Null   Missing   Special   characters   ...   ## 11.   Required and   Optional   Fields   For every   required   field:   ...   Valid value   Missing field   Null   Empty value   where   applicable   Invalid type   Invalid format   Boundary value   Out-of-range   value   ...   For optional   fields:   ...   Field omitted   Valid value   Invalid value			
--	----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	<pre>Null if allowed Boundary values '''  ## 12. Request Body Testing  For request bodies, generate tests for:  ''' Valid request Missing required field Multiple missing fields Optional field omitted Null field Empty field Wrong data type Invalid format Boundary values Invalid enum Malformed JSON Empty object Empty body Unexpected additional field '''</pre>			
--	-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

```
13. State
Transition
Testing

State
transition
testing is
mandatory for
selected
endpoints that
operate on
stateful
resources.

For FR-10
order APIs,
consider:

'''
pending
confirmed
shipping
delivered
'''

Normal
lifecycle:

'''
pending
 |
 v
confirmed
 |
 v
shipping
 |
 v
delivered
'''
```

Generate tests  
for valid and  
invalid  
transitions  
according to  
the API  
specification.

#### ## 14. FR-10 Cancellation Rules

For an order  
cancellation  
endpoint,  
generate tests  
for applicable  
states:

...

Cancel pending  
order

Cancel  
confirmed  
order

Cancel  
shipping order

Cancel  
delivered  
order

Cancel already  
cancelled  
order

Repeated  
cancellation

Invalid  
cancellation  
transition  
...

	Each state test should specify:  <pre>''' Initial State Action Expected Status Code Expected Response Expected Final State '''</pre>  Do not invent cancellation rules.   If the API specification does not define the rule, mark:  <pre>''' Not specified by API contract '''</pre>  ## 15. Security Testing   Security testing must cover the following			
--	----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	<pre>categories where applicable:  ...  SEC-01 - SQL Injection SEC-02 - XSS SEC-03 - IDOR SEC-04 - Authentication Bypass SEC-05 - Role Escalation SEC-06 - Parameter Manipulation SEC-07 - Malformed / Abusive Input ...  ## 16. SEC-01 - SQL Injection  Test applicable input locations:  ...  Path parameters Query parameters Request body Search parameters IDs</pre>			
--	---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

```
Example:
```

```
```\n' OR '1'='1\n1 UNION SELECT\n*\n```\n
```

```
## 17. SEC-02\n- XSS
```

Test
user-controlled
input.

```
Example:
```

```
```\n<script>alert(\n1)</script>\n```\n
```

Verify that  
the API  
handles the  
input  
according to  
the contract  
and does not  
incorrectly  
expose unsafe  
content.

```
18. SEC-03\n- IDOR
```

	<pre>For resources belonging to users, test:  ...  User A token + User B resource ID ...  Verify that unauthorized access or modification is rejected.  ## 19. SEC-04 - Authentication Bypass  Test:  ...  No token Empty token Invalid token Malformed token Expired token Wrong authentication scheme ...  ## 20. SEC-05 - Role Escalation</pre>			
--	----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

Test  
unauthorized  
roles against  
protected  
operations.

Examples:

```

Normal user →
admin
operation

Wrong role
Missing role
Manipulated
role
```

## 21. SEC-06  
- Parameter  
Manipulation

Test  
security-sensi  
tive  
parameters  
such as:

```

ID
User ID
Role
Status
Price
Ownership
Permission
```

	<pre>## 22. SEC-07 - Malformed / Abusive Input  Test:  ...  Very long input Malformed JSON Unexpected Content-Type Special characters Unicode Unexpected fields Invalid encoding ...  ## 23. Response Schema Validation  Every documented response schema must be tested.  Validate:  ...  HTTP status code Content-Type Response shape</pre>			
--	-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	Required properties   Property names   Property types   Nested objects   Arrays   Enums   Nullable fields   Additional properties   ...   The response must match the API specification exactly when the specification defines an exact schema.   ## 24. Error Schema Validation   For every documented error response, validate:   ...   Status code   Content-Type   Response body   Required error fields   Field types			
--	------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	Error schema ```   ## 25. Test Case Structure   Every test case must contain:  ```   Test Case ID Test Case API Method Category Objective Test Data Preconditions Expected Result Expected Status Code Schema Validation Security ID Initial State Expected Final State ```   Fields that are not applicable should contain:  ```   N/A ```			
--	---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	<pre>## 26. Test Case IDs  Use:  ...  TC-{API_IDENTIFI FIER}-{NUMBER} ...  Example:  ...  TC-CANCEL-001 TC-CANCEL-002 TC-CANCEL-003 ... TC-CANCEL-035 ...  IDs must be unique.  ## 27. Test Coverage Validation  After generation, calculate coverage for the selected endpoint.  Check:  ...  Parameter coverage</pre>			
--	-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

	Domain partition coverage Boundary coverage Required field coverage Optional field coverage Data type coverage State transition coverage Cancellation coverage Security coverage Success schema coverage Error schema coverage ...  ## 28. Coverage Repair  If fewer than 35 test cases exist:  ...  WHILE testCaseCount &lt; 35:  missingCoverag e ←			
--	------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--	--	--

```
IdentifyMissingCoverage()
```

```
additionalTests ←
GenerateTests(
 APIContract,
 missingCoverage
)
```

```
Add(additionalTests)
```

```
RemoveDuplicates()
```

```
END WHILE
```
```

The agent should prioritize missing coverage rather than creating meaningless duplicates.

```
## 29.  
Duplicate  
Removal
```

| | | | | |
|--|---|--|--|--|
| | <pre>Remove tests with the same testing purpose. Compare: ''' Purpose Input partition Expected behavior State Security category ''' Different values should not automatically be treated as different tests if they belong to the same equivalence partition. ## 30. Final Test Validation Before output, validate every test: ''' Unique ID</pre> | | | |
|--|---|--|--|--|

| | | | | |
|--|--|--|--|--|
| | <pre>Correct endpoint Correct HTTP method Valid test data Explicit expected result Expected status code Correct category Contract traceability No unsupported assumptions No duplicate purpose ''' ## 31. Output: testcases.md Generate: ''' testcases.md ''' Structure: ''' # API Test Cases ## 1. Selected Endpoint</pre> | | | |
|--|--|--|--|--|

| | | | | |
|--|---|--|--|--|
| | <pre>## 2. API Contract ## 3. Test Design Strategy ## 4. Coverage Summary ## 5. Test Cases Test Case ID Test Case API Method Category Objective Test Data Preconditions Expected Result Expected Status Schema Validation Security ID Initial State Expected Final State --- --- --- - -- --- --- --- --- --- --- - -- --- --- --- ## 6. Coverage Matrix</pre> | | | |
|--|---|--|--|--|

```
## 7.
Assumptions
and
Limitations
```

Only selected
endpoints must
appear in this
file.

32. Output:
tests.json

Generate:
```
tests.json
```

Example:
```json
{
  "api": {
    "method":
"PUT",
    "path":
"/api/orders/{
id}/cancel"
  },

"totalTestCase
s": 35,
  "testCases":
[
  {
    "id":
"TC-CANCEL-001
",
```

```
        "name":
"Cancel valid
pending
order",

        "method":
"PUT",
        "path":
"/api/orders/{
id}/cancel",

        "category":
"Positive",

        "objective":
"Verify
cancellation
of a valid
order",

        "testData": {
            "id":
1
        },

        "preconditions
": [
            "Valid
authenticated
user",
            "Order
is
cancellable"
        ],

        "expectedStatu
s": 200,

        "expectedResul
```

```
t": "Order is cancelled",  
  
"schemaValidation": true,  
  
"securityId": null,  
  
"initialState": "pending",  
  
"expectedFinalState":  
  "cancelled"  
    }  
  ]  
}  
```
```

The actual file must contain all generated test cases.

```
33. Output:
Postman
Collection
```

```
Generate:
```

```
```
```

```
postman_collection.json  
```
```

```
Use Postman
Collection
Schema v2.1.
```

Example:

```
```json
{
  "info": {
    "name":
"AI Generated
API Tests",
    "schema":
"https://schem
a.getpostman.c
om/json/collec
tion/v2.1.0/co
llection.json"
  },
  "item": []
}
```
```

Create one  
Postman  
request for  
every test  
case.

Each request  
should  
contain:

...

Request name  
HTTP method  
URL  
Path variables  
Query  
parameters  
Headers  
Authentication  
Request body

|  |                                                                                                                                                                                                                                                                                                                                                                                  |  |  |  |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|
|  | <pre>Postman test scripts ```  ## 34. Postman Assertions  For status validation:  ```javascript pm.test("Status code is expected", function () {  pm.expect(pm.response.code). to.eql(200); }); ```  For response schema validation, validate the required properties and types.  Example:  ```javascript const response = pm.response.json();  pm.test("Response contains</pre> |  |  |  |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|

```
id", function
() {

pm.expect(resp
onse).to.have.
property("id")
;
});
```

```
pm.test("id is
a number",
function () {

pm.expect(resp
onse.id).to.be
.a("number");
});
````
```

When a JSON
schema exists,
prefer JSON
Schema
validation.

35.
Environment
Variables

Use reusable
Postman
variables:

```
````

{{baseUrl}}
{{token}}
{{userToken}}
{{adminToken}}
{{orderId}}
````
```

```
Never
hard-code
secrets.

## 37. Example
Usage

The user
provides:

...

Specification:
openapi.yaml

Endpoints:
POST
/api/forgot-password
PUT
/api/orders/{id}/cancel
...

The agent must
generate:

...

testcases.md

    Forgot
    Password
    >= 35
    tests

    Cancel
    Order
    >= 35
    tests
```

```
tests.json
```

```
    Forgot  
Password tests  
    Cancel  
Order tests
```

```
postman_collec  
tion.json
```

```
    Forgot  
Password  
requests  
    Cancel  
Order requests  
````
```

```
The agent must
NOT generate
tests for:
```

```
````  
/api/login  
/api/register  
/api/products  
/api/cart  
/api/checkout  
````
```

```
unless the
user
explicitly
selects those
endpoints.
```

```
38. Final
Validation
Checklist
```

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                         |  |  |  |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|
|  | <pre>Before completing the task:  ...  [ ] API specification parsed [ ] Requested endpoints identified [ ] Only requested endpoints processed [ ] API contract extracted [ ] Every applicable parameter analyzed [ ] Domain partitions covered [ ] Boundary values covered [ ] Required fields covered [ ] Optional fields covered [ ] Data types covered [ ] State transitions covered where applicable [ ] FR-10 states covered</pre> |  |  |  |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |  |  |  |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|
|  | <div>where<br/>applicable<br/>[ ]<br/>Cancellation<br/>rules covered<br/>where<br/>applicable<br/>[ ] SEC-01<br/>covered where<br/>applicable<br/>[ ] SEC-02<br/>covered where<br/>applicable<br/>[ ] SEC-03<br/>covered where<br/>applicable<br/>[ ] SEC-04<br/>covered where<br/>applicable<br/>[ ] SEC-05<br/>covered where<br/>applicable<br/>[ ] SEC-06<br/>covered where<br/>applicable<br/>[ ] SEC-07<br/>covered where<br/>applicable<br/>[ ] Success<br/>response<br/>schema covered<br/>[ ] Error<br/>response<br/>schema covered<br/>[ ] &gt;= 35<br/>tests per<br/>selected<br/>endpoint<br/>[ ] Duplicate<br/>tests removed</div> |  |  |  |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|

|  |                                                                                                                                                                                     |  |  |  |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|
|  | <pre>[ ] Unique test IDs [ ] testcases.md generated [ ] tests.json generated [ ] postman_collec tion.json generated [ ] Postman Collection v2.1 [ ] No hard-coded secrets ```</pre> |  |  |  |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|

#### 4. Summary of AI Accuracy

Aggregate the verdicts from Section 3 and complete the table below.

| Metric                                      | Count | Percentage |
|---------------------------------------------|-------|------------|
| <b>Total AI-generated artifacts audited</b> | 4     |            |
| <b>VALID (correct, accepted as-is)</b>      | 1     | 25%        |
| <b>INVALID (wrong; rejected)</b>            | 0     | 0%         |
| <b>INCOMPLETE (acceptable after edits)</b>  | 3     | 75%        |

#### 5. Conclusion — When should AI be used (or not)?

Write 80–150 words describing patterns you observed. Where did AI shine? Where did AI fail? What is your recommendation for using AI in this kind of work in the future?

AI was effective at accelerating repetitive API testing tasks, especially generating large test-case sets, organizing domain partitions, security scenarios, HTTP method checks, and response-schema validations. It also helped create structured Markdown and Postman artifacts consistently. However, AI sometimes introduced assumptions that were not explicitly supported by the API specification or the SUT, such as expecting rate limiting with a 429 response or assuming authorization rules that were not clearly defined. Some security cases also required human judgment to determine whether they were applicable and what the expected result should be. Therefore, AI should be used as a test-design assistant rather than a replacement for the tester. In future work, I would use AI for initial test generation and artifact preparation, then manually validate every case against the API contract, SUT behavior, and testing principles.

## 6. Mandatory Disclosure (paste verbatim)

*"Test cases, test scripts, and supporting test artifacts were initially generated by Cursor AI and ChatGPT; I reviewed and modified the generated test cases, Postman collections, and test specifications, and added additional test cases covering authorization headers, invalid authorization, HTTP method enforcement, and explicit success/error response-schema validation. The AI Audit Report and the final test summary were reviewed and completed by me. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category."*

### Signature

|                                |                                                                                                              |
|--------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Student name (printed):</b> | LE GIA BAO                                                                                                   |
| <b>Student ID:</b>             | 23127325                                                                                                     |
| <b>Class / Cohort:</b>         | KIEM THU PHAN MEM - 23KTPM2                                                                                  |
| <b>Course:</b>                 | CS423 / CSC13003 – Software Testing                                                                          |
| <b>Instructor:</b>             | Dr. Lam Quang Vu / Dr. Tran Duy Hoang / MSc. Tran Thi Bich Hanh / MSc. Truong Phuoc Loc / MSc. Ho Tuan Thanh |
| <b>Date:</b>                   | 30/8/2026                                                                                                    |
| <b>Signature:</b>              | LE GIA BAO                                                                                                   |

### References

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.
- DeepEval & Promptfoo documentation — testing frameworks for LLM systems.